

一、线性表的定义与基本概念

- **定义：**线性表是具有相同数据类型的 $n(n \geq 0)$ 个数据元素的有限序列。
- **关键概念：**
 - n 为表长，当 $n = 0$ 时，说明该线性表是一个空表。
 - 线性表中元素的**位序**从 1 开始计算（注意与编程语言中数组下标从 0 开始区分开来）。
 - 除第一个元素外，每个元素有且仅有一个直接前驱；除最后一个元素外，每个元素有且仅有一个直接后继。
- **基本操作：**数据结构的基本操作通常可以总结为“创、销、增、删、改、查”。具体包括初始化 (InitList)、销毁 (DestroyList)、插入 (ListInsert)、删除 (ListDelete)、按位查找 (GetElem)、按值查找 (LocateElem)、求表长 (Length) 等操作。

操作分类	常见函数名 (C/C++风格)	操作名称	功能描述与参数说明
创	InitList(&L)	初始化表	构造一个空的线性表 L ，并为其分配初始的内存空间。
销	DestroyList(&L)	销毁表	销毁线性表 L ，并释放线性表 L 所占用的全部内存空间。

操作分类	常见函数名 (C/C++风格)	操作名称	功能描述与参数说明
增	<code>ListInsert(&L, i, e)</code>	插入操作	在表 L 中的第 i 个位置（位序，从1开始计算）插入指定的新元素 e 。原有及其后面的元素顺次后移或修改指针。
删	<code>ListDelete(&L, i, &e)</code>	删除操作	删除表 L 中第 i 个位置的元素，并通常用变量 e （引用传入）返回被删除元素的值。
查	<code>LocateElem(L, e)</code>	按值查找	在表 L 中查找具有给定关键字值 e 的元素，通常返回该元素在表中的 位序 （第几个元素）。
查	<code>GetElem(L, i)</code>	按位查找	获取并返回表 L 中第 i 个位置（位序）的元素的值。
其他	<code>Length(L)</code>	求表长	返回线性表 L 当前的长度，即表中数据元素的总个数。
其他	<code>Empty(L)</code>	判空操作	判断线性表 L 是否为空。若 L 为空表（长度为0），则返回 <code>true</code> ，否则返回 <code>false</code> 。
其他	<code>PrintList(L)</code>	输出操作	遍历并按前后次序输出线性表 L 中的所有元素值。

二、顺序表 (顺序存储)

-

定义：用顺序存储的方式实现线性表，即把逻辑上相邻的元素存储在物理位置上也相邻的存储单元中。

- **实现方式：**

-

静态分配：使用大小固定的静态数组存放数据元素，最大长度一旦确定就无法改变。

-

动态分配：使用动态数组实现。当容量存满时，可通过 `malloc` 动态申请一片更大的连续存储空间，然后将原数据复制到新区域，并用 `free` 释放原内存区域。

- **特点：**

-

优点：支持随机访问（随机存取），可以在 $O(1)$ 时间内立即找到第 i 个元素；结点的存储密度高。

-

缺点：拓展容量不方便（动态分配的时间开销也较大）；插入和删除操作不方便，需要移动大量元素。

- **时间复杂度分析：**

-

插入与删除操作：由于需要移动元素，平均情况和最坏情况下的时间复杂度均为 $O(n)$ 。

-

按位查找：得益于连续存放的特性，时间复杂度为 $O(1)$ 。

-

按值查找：需要逐个遍历对比，平均时间复杂度为 $O(n)$ 。

三、 链表 (链式存储)

1. 单链表

-

定义：每个结点除了存放数据元素本身外，还要存储指向下一个节点的指针。

- **头结点机制：**

-

带头结点：在第一个数据结点前附加一个不存数据的头结点。这样对第一个数据结点的插入/删除操作与其他结点逻辑一致，写代码更方便，不易出错。

○

不带头结点：对首结点的处理和空表的处理需要用到不同的代码逻辑，编写更麻烦。

- **插入与删除操作：**

○

按位序操作：必须从头指针出发遍历找到第 $i - 1$ 个结点后进行插入或删除，平均时间复杂度为 $O(n)$ 。

○

指定结点的前插操作：为了避免寻找前驱结点的 $O(n)$ 遍历，可以在该结点之后插入新结点，然后将两者的数据域进行交换（“偷天换日”法），使时间复杂度降为 $O(1)$ 。

•

查找操作：单链表不具备随机访问特性，无论是按位查找还是按值查找，都只能依次扫描，时间复杂度均为 $O(n)$ 。

- **单链表的建立：**

○

尾插法：通过维护一个表尾指针不断在尾部追加结点，生成的链表顺序与输入一致。

○

头插法：始终在头结点之后插入新结点，这种方法可用于实现单链表的逆置。

2. 双链表

•

特点：每个结点既有指向后继的 `next` 指针，也有指向前驱的 `prior` 指针，从而实现了双向检索（可进可退）。

-

操作要点：插入和删除时必须仔细维护 `prior` 和 `next` 指向，需要注意修改指针的先后顺序以防止断链发生。

3. 循环链表

-

循环单链表：将单链表表尾结点的 `next` 指针指向头结点，使得从任意一个结点出发都可以遍历到整个链表。

-

循环双链表：表头结点的 `prior` 指向表尾结点，而表尾结点的 `next` 指向表头结点。

4. 静态链表

-

定义：在内存中分配一整片连续空间，用数组的方式实现链表，以数组下标（游标）充当“指针”来记录下一个结点的位置。

-

优缺点：增、删操作仅需修改游标，不需要移动元素；缺点是容量固定不可变，且不支持随机存取。

-

应用场景：适用于不支持指针的低级语言，或者数据元素数量固定不变的场景（如操作系统的 FAT 文件分配表）。

四、 顺序表与链表的综合对比

-

空间与扩容：顺序表要求大片连续内存，且改变容量不方便；链表支持离散分配空间，弹性好，易于扩容。

- **操作效率对比：**

-

顺序表的优势在于“查”：由于支持随机存取，按位查找非常快。

-

链表的优势在于“增、删”：只需修改指针，不需要大规模移动元素数据。

- **实际选型：**

- 当**表长难以预估**、或者经常需要执行**增加和删除**操作时，优先使用**链表**。

- 当**表长较稳定可预估**、且主要执行**查询**（搜索）操作时，优先使用**顺序表**。